



Orviohub User Profile: Full Implementation Plan

Executive Summary

Orviohub's personal profile system enables users to manage their identity, security, preferences, and data privacy across all workspaces. This implementation covers **security & authentication** (2FA, password, sessions), **contact management** (email, phone), **notification preferences**, **workspace memberships**, and **privacy controls** (data export, account deletion). The design follows GitHub's model: personal profile is user-owned, workspace admins cannot access personal security data. ^{[1] [2] [3] [4]}

1. System Architecture

1.1 Core Principles

- **User-owned identity:** Personal profile belongs to the user, not workspace admins ^[1]
- **Workspace separation:** Profile (accounts.orviohub.localhost) separate from workspace settings ^{[3] [1]}
- **Re-authentication:** Sensitive actions (password change, email change, 2FA disable) require current password or recent auth ^{[5] [6] [3]}
- **Audit everywhere:** All profile changes create audit logs ^[1]
- **NDPR compliance:** Data export, deletion, consent management aligned with Nigeria Data Protection Act ^{[7] [1]}

1.2 Data Model (Convex Schema)

```
// users - core identity (existing, enhanced)
users: defineTable({
  email: v.string(),
  emailNormalized: v.string(),
  emailVerifiedAt: v.optional(v.number()),
  firstName: v.optional(v.string()),
  lastName: v.optional(v.string()),
  displayName: v.optional(v.string()),
  avatarUrl: v.optional(v.string()),
  phone: v.optional(v.string()),
  phoneVerifiedAt: v.optional(v.number()),
  country: v.optional(v.string()),
  timezone: v.string(), // default: "Africa/Lagos"
```

```

    language: v.string(), // default: "en"
    status: v.string(), // "active" | "deletion_requested" | "deleted"
    createdAt: v.number(),
    updatedAt: v.number(),
    lastLoginAt: v.optional(v.number()),
  })
  .index("by_email", ["emailNormalized"])
  .index("by_status", ["status"])

// authIdentities - OAuth/social logins
authIdentities: defineTable({
  userId: v.id("users"),
  provider: v.string(), // "google" | "facebook" | "email"
  providerSubject: v.string(), // provider's user ID
  providerEmail: v.optional(v.string()),
  providerEmailVerified: v.boolean(),
  createdAt: v.number(),
  lastUsedAt: v.optional(v.number()),
  linkedAt: v.number(),
  metadata: v.optional(v.any()),
})
  .index("by_user", ["userId"])
  .index("by_provider_subject", ["provider", "providerSubject"])

// totpSecrets - 2FA TOTP secrets (encrypted)
totpSecrets: defineTable({
  userId: v.id("users"),
  secretEncrypted: v.string(), // AES-256-GCM encrypted
  backupCodesHashed: v.array(v.string()), // bcrypt hashes
  enabledAt: v.optional(v.number()),
  disabledAt: v.optional(v.number()),
  createdAt: v.number(),
})
  .index("by_user", ["userId"])

// userSessions - active sessions (enhanced from auth plan)
userSessions: defineTable({
  userId: v.id("users"),
  sessionHash: v.string(), // SHA-256 hash of session token
  deviceId: v.optional(v.string()),
  deviceName: v.optional(v.string()), // "Chrome on macOS"
  browser: v.optional(v.string()), // "Chrome 127"
  operatingSystem: v.optional(v.string()), // "macOS Sonoma"
  ipAddress: v.string(),
  approximateLocation: v.optional(v.string()), // "Abuja, NG" (from IP geolocation)
  authenticationMethod: v.string(), // "password" | "google" | "2fa"
  createdAt: v.number(),
  lastActiveAt: v.number(),
  expiresAt: v.number(),
  revokedAt: v.optional(v.number()),
})
  .index("by_user", ["userId"])
  .index("by_user_active", ["userId", "revokedAt"]) // active = revokedAt is null

// userPreferences - UI/product preferences
userPreferences: defineTable({

```

```

    userId: v.id("users"),
    theme: v.string(), // "light" | "dark" | "system"
    language: v.string(), // "en" | "fr" etc.
    timezone: v.string(),
    dateFormat: v.string(), // "DD/MM/YYYY" | "MM/DD/YYYY"
    numberFormat: v.string(), // "1,000.00" | "1.000,00"
    marketingEmailsEnabled: v.boolean(),
    productEmailsEnabled: v.boolean(),
    securityEmailsEnabled: v.boolean(),
    updatedAt: v.number(),
  })
  .index("by_user", ["userId"])

// userConsents - GDPR/NDPR consent tracking
userConsents: defineTable({
  userId: v.id("users"),
  consentType: v.string(), // "marketing_emails" | "data_processing"
  version: v.string(), // "1.0" (track consent version)
  granted: v.boolean(),
  grantedAt: v.number(),
  withdrawnAt: v.optional(v.number()),
  source: v.string(), // "signup" | "settings"
})
  .index("by_user", ["userId"])
  .index("by_user_consent", ["userId", "consentType"])

// dataExportRequests - data export jobs
dataExportRequests: defineTable({
  userId: v.id("users"),
  status: v.string(), // "pending" | "processing" | "completed" | "failed"
  requestedAt: v.number(),
  completedAt: v.optional(v.number()),
  expiresAt: v.number(), // download link expires after 7 days
  storageReference: v.optional(v.string()), // S3/Convex storage file ID
})
  .index("by_user", ["userId"])

// accountDeletionRequests - deletion queue
accountDeletionRequests: defineTable({
  userId: v.id("users"),
  status: v.string(), // "requested" | "cooling_off" | "scheduled" | "completed" | "cancelled"
  reason: v.optional(v.string()),
  requestedAt: v.number(),
  scheduledDeletionAt: v.optional(v.number()), // cooling-off period end
  cancelledAt: v.optional(v.number()),
  completedAt: v.optional(v.number()),
})
  .index("by_user", ["userId"])

// userAuditLogs - personal security audit
userAuditLogs: defineTable({
  userId: v.id("users"),
  eventType: v.string(), // "password.changed" | "2fa.enabled" | "session.revoked"
  targetType: v.optional(v.string()), // "session" | "email" | "totp"
  targetId: v.optional(v.string()),
  severity: v.string(), // "low" | "medium" | "high"
})

```

```

    metadata: v.optional(v.any()), // { oldEmail, newEmail, deviceId }
    requestId: v.optional(v.string()),
    ipAddress: v.optional(v.string()),
    userAgent: v.optional(v.string()),
    createdAt: v.number(),
  })
  .index("by_user", ["userId"])
  .index("by_user_created", ["userId", "createdAt"])

```

2. User Journeys & Stories

2.1 Primary User Personas

Persona	Description	Goals
Code (Owner)	Orviohub founder, owns 2 workspaces, security-conscious	Enable 2FA, manage sessions, export data, control notifications
Mary (Worker)	Sales attendant at Code X Stores, no workspace ownership	Change password, view org access, leave workspace, download data
John (New User)	Just signed up, invited to Code X Stores	Set up profile, enable 2FA, understand workspace access

2.2 Journey 1: User Enables 2FA (TOTP)

Actor: Code (security-conscious user)

Precondition: Logged into `accounts.orviohub.localhost`, on Security settings page

Step-by-Step Flow

Step 1: Navigate to 2FA Settings

- Code clicks "Security" in sidebar
- Frontend calls `GET /v1/users/me/security` (returns 2FA status, active sessions count)
- UI shows: "Two-Factor Authentication: **Disabled**" + "Enable" button

Step 2: Initiate 2FA Setup

- Code clicks "Enable Two-Factor Authentication"
- Modal opens: "Set up Two-Factor Authentication"
- **Warning:** "You'll need an authenticator app like Google Authenticator or Authy"
- **Button:** "Get Started"

Step 3: Re-authenticate (Security Boundary)

- Backend requires re-authentication before enabling 2FA ^{[4] [6] [3]}
- Prompt: "Enter your password to continue"

- Code enters current password
- POST /v1/users/me/security/2fa/verify-password
- Backend validates password (rate-limited, audit logged)
- Returns: { verified: true, sessionValid: true }

Step 4: Generate TOTP Secret

Backend (POST /v1/users/me/security/2fa/start):

1. Generate cryptographically secure secret (20 bytes, base32 encoded) ^{[2] [8] [4]}

```
const secret = speakeasy.generateSecret({
  length: 20,
  name: `Orviohub:${code.email}`,
  issuer: "Orviohub"
});
```

2. Encrypt secret with AES-256-GCM (KMS-managed key) ^{[8] [9] [4]}

```
const encrypted = encrypt(secret.base32, KMS_KEY);
```

3. Generate 10 backup codes (8-12 random 8-char codes) ^{[10] [2]}

```
const backupCodes = Array.from({ length: 10 }, () =>
  crypto.randomBytes(4).toString('hex')
);
```

4. Hash backup codes with bcrypt (store only hashes) ^[10]

```
const hashedCodes = await Promise.all(
  backupCodes.map(code => bcrypt.hash(code, 12))
);
```

5. Store in totpSecrets table (status: "pending", not enabled yet)

```
totpSecrets.insert({
  userId: code.userId,
  secretEncrypted: encrypted,
  backupCodesHashed: hashedCodes,
  createdAt: Date.now()
});
```

6. Write audit log: 2fa.setup_initiated (severity: high)

7. Return: { secretUrl: "otpauth://totp/...", backupCodes: ["abc12345", ...] } (plain backup codes shown once)

Step 5: Display QR Code + Backup Codes

UI shows:

- QR code (generated from secretUrl using qrcode library) ^{[2] [4]}
- Manual entry key: "Orviohub (code@example.com) - Key: ABCD EFGH IJKL MNOP"

- **Backup codes (one-time display):**

Save these backup codes in a safe place.
You can only see them once.

```
abc12345    def67890    ghi11223
jkl144556   mno77889    pqr99000
stu11223    vwx33445    yza55667
bcd77889
```

- **Checkbox:** "I've saved my backup codes"
- **Button:** "Continue" (disabled until checkbox checked)

Step 6: Verify TOTP Code

- Code opens Google Authenticator, scans QR code
- App shows 6-digit code (refreshes every 30s)
- Code enters code: "123456"
- POST /v1/users/me/security/2fa/verify

```
{ "totpCode": "123456" }
```

- **Backend:**

1. Decrypt secret from DB

2. Verify TOTP with ± 1 window (allows clock skew) ^[8] ^[2]

```
const valid = speakeasy.totp.verify({
  secret: decryptedSecret,
  encoding: 'base32',
  token: totpCode,
  window: 1
});
```

3. Rate-limit: max 5 failed attempts per 15 min ^[11] ^[4] ^[2]

4. If valid:

- Update `totpSecrets.enabledAt = Date.now()`
- Write audit log: `2fa.enabled (severity: high)`
- Send security email: "2FA enabled on your account"
- Return: { `success: true` }

5. If invalid:

- Increment failed attempt counter
- Return: { `error: "Invalid code. 4 attempts remaining."` }

Step 7: Completion

- UI shows: "✓ Two-Factor Authentication Enabled"

- **Next steps:**
 - "Download backup codes" (PDF)
 - "Set up recovery phone" (future feature)
 - "Continue to dashboard"
- Code's future logins require password + TOTP code

2.3 Journey 2: User Changes Password

Actor: Mary (wants to update password)

Precondition: Logged in, on Security → Password page

Step-by-Step Flow

Step 1: Navigate to Password Change

- Mary clicks "Security" → "Password"
- UI shows: "Change Password" form

Step 2: Enter Passwords

Form fields:

- **Current password** (required)
- **New password** (required, min 8 chars, strength meter)
- **Confirm new password** (required)
- **Password requirements** (display):
 - ✓ At least 8 characters
 - ✓ One uppercase letter
 - ✓ One number
 - ✓ One special character

Step 3: Backend Validation

POST /v1/users/me/password/change

```
{
  "currentPassword": "OldPass123!",
  "newPassword": "NewSecure456!"
}
```

Backend checks:

1. ✓ Current password matches (bcrypt compare)
2. ✓ New password ≠ current password
3. ✓ New password meets strength requirements (zxcvbn score ≥ 3)

- ✓ Rate-limit: max 5 attempts per hour^[12] ^[13]
- ✓ Re-authentication confirmed (current password serves as re-auth)^[3] ^[5]

Step 4: Update Password

Backend transaction:

1. Hash new password with bcrypt (cost 12)^[5]

```
const hashed = await bcrypt.hash(newPassword, 12);
```

2. Update `users.passwordHash = hashed`

3. Revoke all other sessions (security boundary)^[14] ^[12]

```
userSessions.update(  
  { userId: mary.userId, revokedAt: Date.now() },  
  { filter: { sessionHash: currentSessionHash } } // keep current session  
);
```

4. Write audit log: `password.changed` (severity: high)

5. Send security email: "Password changed on your account"^[12]

- Subject: "Password changed - Orvionhub"
- Body: "Your password was changed successfully. If you didn't make this change, contact support."

6. Return: `{ success: true }`

Step 5: UI Response

- Toast: "✓ Password changed successfully"
- **Session warning:** "You've been logged out of other devices. Sign in again on those devices with your new password."
- Redirect to dashboard (current session remains active)

2.4 Journey 3: User Views & Manages Active Sessions

Actor: Code (detects suspicious login)

Precondition: Logged in, on Security → Sessions page

Step-by-Step Flow

Step 1: Load Sessions List

- Code navigates to "Security" → "Active Sessions"
- Frontend calls `GET /v1/users/me/sessions`
- Backend queries `userSessions.by_user` (filters `revokedAt` is null)
- Returns list of active sessions

Step 2: Display Sessions

UI table:

Device	Browser	Location	IP	Last Active	Status
MacBook Pro	Chrome 127	Abuja, NG	105.112.x.x	Now	Current device
iPhone 14	Safari 17	Lagos, NG	41.184.x.x	2 hours ago	Sign out
Windows PC	Firefox 128	Unknown	197.210.x.x	5 days ago	Sign out

Step 3: Identify Suspicious Session

- Code sees "Windows PC - Firefox" (doesn't recognize)
- Clicks "Sign out" button on that row
- Confirmation modal: "Sign out of this device?"
 - Device: Windows PC (Firefox 128)
 - Location: Unknown (197.210.x.x)
 - Last active: 5 days ago
 - **Warning:** "This will revoke the session immediately"

Step 4: Revoke Session

POST /v1/users/me/sessions/:sessionId/revoke

Backend:

1. Verify session belongs to authenticated user
2. Update `userSessions.revokedAt = Date.now()`
3. Invalidate session token (remove from cache/Redis)
4. Write audit log: `session.revoked (severity: medium)`

```
userAuditLogs.insert({
  userId: code.userId,
  eventType: "session.revoked",
  targetType: "session",
  targetId: sessionId,
  severity: "medium",
  metadata: { deviceId, browser, ipAddress },
  ipAddress: request.ip,
  createdAt: Date.now()
});
```

5. Send security email (optional): "You signed out of a device"
6. Return: `{ success: true }`

Step 5: UI Update

- Session row updates: "Signed out" (grayed out)
- Toast: "✓ Session revoked"

- Code can now:
 - **Sign out all other sessions:** POST /v1/users/me/sessions/revoke-all (keeps current)
 - **Report suspicious activity:** Links to support form

2.5 Journey 4: User Changes Email Address

Actor: Mary (changed email provider)

Precondition: Logged in, on Profile → Contact page

Step-by-Step Flow

Step 1: Initiate Email Change

- Mary clicks "Contact" in sidebar
- Current email: "mary.old@gmail.com" (verified badge)
- Clicks "Change email"
- Modal: "Change your email address"

Step 2: Re-authenticate + Enter New Email

Form:

- **Current password** (required, for re-auth)^[3] ^[5]
- **New email** (required, format validation)
- **Confirm new email** (required, must match)

Step 3: Backend Validation

POST /v1/users/me/email/change

```
{
  "currentPassword": "MaryPass123!",
  "newEmail": "mary.new@outlook.com"
}
```

Backend checks:

1. ✓ Current password matches
2. ✓ New email format valid (RFC 5322)
3. ✓ New email not already registered (unique constraint)
4. ✓ Normalize email: lowercase, trim whitespace

Step 4: Send Verification Email

Backend:

1. Generate verification token (CSPRNG, 32 bytes, URL-safe base64)^[13]

```
const token = crypto.randomBytes(32).toString('base64url');
```

2. Hash token (SHA-256), store in temporary table^[13]

```
emailChangeTokens.insert({
  userId: mary.userId,
  tokenHash: sha256(token),
  newEmail: "mary.new@outlook.com",
  expiresAt: Date.now() + 24 * 60 * 60 * 1000, // 24 hours
  createdAt: Date.now()
});
```

3. Send verification email to **new email**^[13]

- Subject: "Verify your new email - Orviohub"
- Body: "Click to verify: mary.new@outlook.com"
- Link: <https://accounts.orviohub.localhost/email/verify?token={token}>

4. Send notification email to **old email**^[1]

- Subject: "Email change requested on your account"
- Body: "Your email is being changed to mary.new@outlook.com. If you didn't request this, contact support immediately."

5. Write audit log: email.change_requested (severity: high)

```
6. Return: { success: true, message: "Verification email sent to mary.new@outlook.com"
}
```

Step 5: User Clicks Verification Link

- Mary checks new email inbox, clicks link
- Redirects to: accounts.orviohub.localhost/email/verify?token=abc123
- Frontend calls POST `/v1/users/me/email/verify` with token

Backend:

1. Hash incoming token, lookup in emailChangeTokens

2. Verify:

- ✓ Token exists
- ✓ Not expired
- ✓ Matches userId from session

3. Update user record:

```
users.update(mary.userId, {
  email: "mary.new@outlook.com",
  emailNormalized: "mary.new@outlook.com",
  emailVerifiedAt: Date.now()
});
```

4. Invalidate token (single-use)^[13]

5. Revoke all sessions (high-risk change)^[1]

```
userSessions.update(
  { userId: mary.userId, revokedAt: Date.now() },
  { filter: { sessionHash: currentSessionHash } } // keep current
);
```

6. Write audit log: email.changed (severity: high)

7. Send confirmation email to both old and new emails^[1]

8. Return: { success: true }

Step 6: Post-Change Experience

- Mary's session remains active (current device)
- Toast: "✓ Email changed to mary.new@outlook.com"
- **Warning:** "You've been logged out of other devices. Sign in again with your new email."
- Future logins use new email

2.6 Journey 5: User Downloads Personal Data (GDPR/NDPR)

Actor: Code (wants to see what data Orviohub stores)

Precondition: Logged in, on Privacy → Data & Privacy page

Step-by-Step Flow

Step 1: Request Data Export

- Code clicks "Privacy" → "Download your data"
- UI shows: "What's included":
 - Personal profile (name, email, phone, preferences)
 - Authentication methods (email, Google, Facebook - no secrets)
 - Workspace memberships (org names, roles, join dates)
 - Notification preferences
 - Security activity summary (login history, 2FA events)
 - Consents granted
- **Button:** "Request export"

Step 2: Create Export Job

POST /v1/users/me/data-export

Backend:

1. Check rate limit: max 1 export per 7 days^[1]

2. Create dataExportRequests row:

```
dataExportRequests.insert({
  userId: code.userId,
```

```
status: "pending",
requestedAt: Date.now(),
expiresAt: Date.now() + 7 * 24 * 60 * 60 * 1000 // 7 days
});
```

3. Trigger async job (Convex action or background worker)

4. Return: { exportId: "exp_123", status: "processing", estimatedTime: "5-10 minutes" }

Step 3: Background Export Job

Async process:

1. Query user data (sanitized, no secrets):

```
const userData = await users.get(code.userId);
const memberships = await orgMembers.by_user({ userId: code.userId });
const preferences = await userPreferences.by_user({ userId: code.userId });
const auditSummary = await userAuditLogs.by_user({ userId: code.userId });
```

2. Compile JSON export:

```
{
  "user": {
    "email": "code@example.com",
    "firstName": "Code",
    "lastName": "X",
    "displayName": "Code X",
    "timezone": "Africa/Lagos",
    "language": "en",
    "createdAt": 1694908800000
  },
  "authMethods": [
    { "provider": "email", "verified": true },
    { "provider": "google", "email": "code@example.com" }
  ],
  "memberships": [
    {
      "organizationName": "Code X Stores",
      "role": "owner",
      "joinedAt": 1694908800000,
      "products": ["inventory", "task_management"]
    }
  ],
  "preferences": {
    "theme": "dark",
    "marketingEmailsEnabled": false,
    "securityEmailsEnabled": true
  },
  "securityActivity": {
    "totalLogins": 142,
    "lastLoginAt": 1694908800000,
    "2faEnabled": true,
    "sessionsRevoked": 3
  },
  "consents": [
```

```
    { "type": "data_processing", "granted": true, "version": "1.0" }  
  ]  
}
```

3. Generate ZIP file (JSON + CSV for audit logs)
4. Upload to secure storage (Convex storage or S3, private bucket)
5. Generate pre-signed download URL (expires in 7 days)
6. Update dataExportRequests:

```
dataExportRequests.update(exportId, {  
  status: "completed",  
  completedAt: Date.now(),  
  storageReference: "s3://orviohub-exports/exp_123.zip"  
});
```

7. Send email notification:

- Subject: "Your Orviohub data export is ready"
- Body: "Download your data (expires in 7 days)" + download link

8. Write audit log: `data.export_completed`

Step 4: User Receives Notification

- In-app notification: "Your data export is ready to download"
- Email: "Download your Orviohub data"
- Code clicks link → downloads ZIP file

Step 5: Download Data

- Code clicks download link (validates token, checks expiry)
- Browser downloads: `orviohub-data-export-2026-09-09.zip`
- Contents:
 - `profile.json` (personal data)
 - `memberships.csv` (workspace access)
 - `security-activity.csv` (audit summary)
 - `README.txt` (explanation of fields)

2.7 Journey 6: User Deletes Account

Actor: Mary (leaving Orviohub)

Precondition: Logged in, on Privacy → Delete Account page

Step-by-Step Flow

Step 1: Initiate Deletion

- Mary clicks "Privacy" → "Delete your account"
- UI shows **warning modal**:
 - "This action is permanent and cannot be undone"
 - "You will lose access to:"
 - All workspace memberships (Code X Stores, etc.)
 - All app data (Inventory, Task Management)
 - Your profile and settings
 - "Business records will be preserved" (employer's data not deleted)
 - **Input**: "Type 'DELETE' to confirm"
 - **Button**: "Request deletion" (disabled until "DELETE" typed)

Step 2: Check Ownership Constraints

POST /v1/users/me/deletion-request

```
{ "confirmationText": "DELETE" }
```

Backend checks:

1. ✓ Confirmation text matches
2. ✗ **Check workspace ownership**: Query `orgMembers.by_user where role="owner"`
 - If Mary owns any workspace:
 - Return error: "You own 1 workspace (Code X Stores). Transfer ownership or delete the workspace before deleting your account."
 - Provide link: "Manage workspaces" → `accounts.orv.iohub.localhost/workspaces`
3. ✓ No active deletion request already
4. ✓ Re-authentication: Require current password (high-risk action) ^[5] ^[1]

Step 3: Create Deletion Request

Backend:

1. Insert `accountDeletionRequests` row:

```
accountDeletionRequests.insert({
  userId: mary.userId,
  status: "cooling_off",
  requestedAt: Date.now(),
  scheduledDeletionAt: Date.now() + 7 * 24 * 60 * 60 * 1000, // 7-day cooling-off
});
```

2. Update user status: `users.status = "deletion_requested"`

3. Write audit log: `account.deletion_requested` (severity: critical)

4. Send confirmation email:

- Subject: "Account deletion requested - Orviohub"
- Body: "Your account will be deleted on {{date}}. Cancel anytime before then."
- **Cancel link:** `accounts.orviohub.localhost/settings/privacy/cancel-deletion`

5. Return: { success: true, scheduledDeletionAt: timestamp }

Step 4: Cooling-Off Period (7 Days)

- Mary receives email: "Account deletion scheduled"
- During 7 days:
 - Can still log in (limited functionality)
 - Can cancel deletion: `POST /v1/users/me/deletion-request/cancel`
 - Banner on all pages: "Your account will be deleted on {{date}}. [Cancel deletion]"

Step 5: Background Deletion Job (After 7 Days)

Scheduled cron job (runs daily):

1. Query `accountDeletionRequests` where:

- `status = "cooling_off" AND scheduledDeletionAt < now`

2. For each request:

- **Preserve business records:** Do NOT delete workspace data, transactions, audit logs^[15]
[1]
- **Anonymize user references:**

```
users.update(userId, {
  email: `deleted_${userId}@orviohub.local`,
  emailNormalized: `deleted_${userId}@orviohub.local`,
  firstName: null,
  lastName: null,
  displayName: "Deleted User",
  avatarUrl: null,
  phone: null,
  status: "deleted"
});
```

- **Revoke all sessions:** `userSessions.update({ userId, revokedAt: now })`
- **Delete personal data:**
 - `authIdentities` (remove OAuth links)
 - `totpSecrets` (remove 2FA secrets)
 - `userPreferences` (delete preferences)
 - `userConsents` (delete consents)
- **Preserve audit logs:** Keep `userAuditLogs` (compliance requirement)

- **Remove from workspace memberships:** Set `orgMembers.status = "removed"` (don't delete rows, preserve history)
- Update `accountDeletionRequests.status = "completed", completedAt = now`

3. Write audit log: `account.deleted` (severity: critical)

4. Send final email (optional): "Your account has been deleted"

Step 6: Post-Deletion

- Mary's account is anonymized, not fully deleted (business records preserved)
- Future login attempts with old email: "Account not found" (no enumeration)
- Workspace owners see "Deleted User" in member lists (preserves referential integrity)

3. API Endpoints (Fastify)

3.1 Personal Profile Routes

```
// GET /v1/users/me
// Get current user profile
→ {
  user: {
    id: "usr_123",
    email: "code@example.com",
    emailVerified: true,
    firstName: "Code",
    lastName: "X",
    displayName: "Code X",
    avatarUrl: "https://cdn.orv.iohub.com/avatars/usr_123.png",
    timezone: "Africa/Lagos",
    language: "en",
    createdAt: 1694908800000
  }
}

// PATCH /v1/users/me
// Update profile fields (name, displayName, timezone, language)
{
  firstName?: string,
  lastName?: string,
  displayName?: string,
  timezone?: string,
  language?: string
}
→ { success: true, user: {...} }

// POST /v1/users/me/avatar
// Upload profile photo (multipart/form-data)
// Returns avatar URL
→ { avatarUrl: string }

// DELETE /v1/users/me/avatar
```

```
// Remove profile photo
→ { success: true }

// PATCH /v1/users/me/contact
// Update contact info (phone, country)
{
  phone?: string,
  country?: string,
  state?: string,
  city?: string
}
→ { success: true }
```

3.2 Security Routes

```
// POST /v1/users/me/password/change
// Change password (requires current password)
{
  currentPassword: string,
  newPassword: string
}
→ { success: true }

// GET /v1/users/me/security/2fa/status
// Check 2FA status
→ {
  totpEnabled: boolean,
  backupCodesRemaining?: number,
  enabledAt?: number
}

// POST /v1/users/me/security/2fa/start
// Start 2FA setup (returns QR code URL + backup codes)
→ {
  secretUrl: string, // otpauth://totp/...
  backupCodes: string[] // plain codes (shown once)
}

// POST /v1/users/me/security/2fa/verify
// Verify TOTP code (final step of 2FA setup)
{
  totpCode: string
}
→ { success: true }

// POST /v1/users/me/security/2fa/disable
// Disable 2FA (requires re-auth + TOTP code)
{
  currentPassword: string,
  totpCode: string
}
→ { success: true }

// GET /v1/users/me/sessions
```

```

// List active sessions
→ {
  sessions: {
    id: string,
    deviceName: string,
    browser: string,
    operatingSystem: string,
    ipAddress: string,
    approximateLocation: string,
    lastActiveAt: number,
    isCurrent: boolean
  }[]
}

// POST /v1/users/me/sessions/:sessionId/revoke
// Revoke specific session
→ { success: true }

// POST /v1/users/me/sessions/revoke-all
// Revoke all other sessions (keep current)
→ { success: true, revokedCount: number }

// GET /v1/users/me/security-activity
// List security audit logs (paginated)
{
  limit?: number,
  cursor?: string,
  eventType?: string,
  startDate?: number,
  endDate?: number
}
→ {
  logs: {
    eventType: string,
    severity: string,
    ipAddress: string,
    userAgent: string,
    createdAt: number,
    metadata?: any
  }[],
  nextCursor?: string
}

```

3.3 Email & Phone Routes

```

// POST /v1/users/me/email/change
// Request email change (sends verification email)
{
  currentPassword: string,
  newEmail: string
}
→ { success: true, message: "Verification email sent" }

// POST /v1/users/me/email/verify

```

```

// Verify email change token
{
  token: string
}
→ { success: true }

// POST /v1/users/me/phone/send-otp
// Send OTP to phone (for verification)
{
  phone: string
}
→ { success: true, otpExpiresIn: 300 } // 5 minutes

// POST /v1/users/me/phone/verify
// Verify phone OTP
{
  phone: string,
  otp: string
}
→ { success: true }

```

3.4 Preferences Routes

```

// GET /v1/users/me/preferences
// Get user preferences
→ {
  theme: "dark",
  language: "en",
  timezone: "Africa/Lagos",
  dateFormat: "DD/MM/YYYY",
  numberFormat: "1,000.00",
  marketingEmailsEnabled: false,
  productEmailsEnabled: true,
  securityEmailsEnabled: true
}

// PATCH /v1/users/me/preferences
// Update preferences
{
  theme?: "light" | "dark" | "system",
  language?: string,
  timezone?: string,
  dateFormat?: string,
  numberFormat?: string,
  marketingEmailsEnabled?: boolean,
  productEmailsEnabled?: boolean,
  securityEmailsEnabled?: boolean
}
→ { success: true }

// GET /v1/users/me/notifications/preferences
// Get notification preferences (per category)
→ {
  security: { email: true, inApp: true },

```

```

    workspace: { email: true, inApp: true },
    product: { email: false, inApp: true },
    billing: { email: true, inApp: true },
    marketing: { email: false, inApp: false }
  }

// PATCH /v1/users/me/notifications/preferences
// Update notification preferences
{
  security?: { email?: boolean, inApp?: boolean },
  workspace?: { email?: boolean, inApp?: boolean },
  product?: { email?: boolean, inApp?: boolean },
  billing?: { email?: boolean, inApp?: boolean },
  marketing?: { email?: boolean, inApp?: boolean }
}
→ { success: true }

```

3.5 Privacy Routes

```

// GET /v1/users/me/data-summary
// Get summary of stored data (counts, categories)
→ {
  profileFields: 12,
  memberships: 2,
  securityEvents: 47,
  consents: 1,
  totalDataSize: "24 KB"
}

// POST /v1/users/me/data-export
// Request data export
→ {
  exportId: "exp_123",
  status: "processing",
  estimatedTime: "5-10 minutes"
}

// GET /v1/users/me/data-export/:exportId
// Check export status
→ {
  exportId: "exp_123",
  status: "completed" | "processing" | "failed",
  downloadUrl?: string, // if completed
  expiresAt?: number
}

// POST /v1/users/me/deletion-request
// Request account deletion
{
  confirmationText: "DELETE",
  reason?: string
}
→ {
  success: true,

```

```

    scheduledDeletionAt: number
  }

// POST /v1/users/me/deletion-request/cancel
// Cancel pending deletion
→ { success: true }

```

3.6 Workspace Membership Routes

```

// GET /v1/users/me/workspaces
// List all workspace memberships
→ {
  workspaces: {
    id: string,
    name: string,
    logoUrl?: string,
    type: string,
    role: "owner" | "admin" | "member",
    products: ["inventory", "task_management"],
    membershipStatus: "active" | "suspended",
    joinedAt: number,
    ownerName?: string
  }[]
}

// POST /v1/users/me/workspaces/:workspaceId/leave
// Leave workspace (if not only owner)
→ { success: true }
// Error if only owner: "You must transfer ownership before leaving"

```

4. UI/UX Specifications

4.1 Profile Settings Layout

URL: `accounts.orv.iohub.localhost/settings`

Sidebar Navigation:

- **Personal** (name, avatar, bio, job title)
- **Contact** (email, phone, address)
- **Security** (password, 2FA, sessions, activity)
- **Notifications** (email/in-app preferences)
- **Preferences** (theme, language, timezone, formats)
- **Workspaces** (list of orgs, roles, leave workspace)
- **Privacy** (data export, delete account, consents)

Layout:

- Left sidebar (200px): Navigation links
- Main content (600px max-width): Form sections
- Top banner: "Last updated: 2 hours ago"

4.2 Security Page Components

4.2.1 Password Section

UI:

Password

Last changed: 3 months ago

[Change password]

Modal: Change Password

Current password: [.....]

New password: [.....]

Confirm password: [.....]

Password requirements:

✓ At least 8 characters

✓ One uppercase letter

✓ One number

✓ One special character

[Cancel] [Change password]

4.2.2 Two-Factor Authentication Section

UI (2FA disabled):

Two-Factor Authentication

Add an extra layer of security to your account.

Status: Disabled

[Enable two-factor authentication]

Modal: Set up 2FA

Step 1: Install an authenticator app

Download Google Authenticator, Authy, or similar.

Step 2: Scan QR code

[QR code image]

Or enter manually:

Key: ABCD EFGH IJKL MNOP

Step 3: Enter verification code

Code: [^123456]

Backup codes:

Save these in a safe place. You can only see them once.

abc12345 def67890 ghi11223
jkl44556 mno77889 pqr99000
stu11223 vwx33445 yza55667
bcd77889

[] I've saved my backup codes

[Cancel] [Continue]

UI (2FA enabled):

Two-Factor Authentication

Status: Enabled (since Sep 1, 2026)

Backup codes: 8 remaining

[Download backup codes] [Disable two-factor authentication]

4.2.3 Active Sessions Section

UI:

Active Sessions

Manage devices where you're signed in.

Current device: MacBook Pro - Chrome 127 (Abuja, NG)

Device	Browser	Location	Last Active	Actions
MacBook Pro	Chrome 127	Abuja, NG	Now	Current device
iPhone 14	Safari 17	Lagos, NG	2 hours ago	[Sign out]
Windows PC	Firefox 128	Unknown	5 days ago	[Sign out]

[Sign out all other sessions]

Modal: Sign out of this device?

Device: Windows PC (Firefox 128)

Location: Unknown (197.210.x.x)

Last active: 5 days ago

This will revoke the session immediately.

[Cancel] [Sign out]

4.2.4 Security Activity Section

UI:

Security Activity

Recent security events on your account.

Filter: [All events ▼] [Last 30 days ▼]

Date	Event	Device	Location	Details
Sep 9, 2026	Password changed	Chrome 127	Abuja, NG	View details
Sep 5, 2026	New device signed in	Safari 17	Lagos, NG	View details
Sep 1, 2026	2FA enabled	Chrome 127	Abuja, NG	View details
Aug 28, 2026	Email changed	Chrome 127	Abuja, NG	View details

[Download full activity log]

4.3 Privacy Page Components

4.3.1 Data Export Section

UI:

Download Your Data

Orvionhub stores the following information about you:

- Personal profile (name, email, phone, preferences)
- Authentication methods (email, Google, Facebook)
- Workspace memberships (organizations, roles, join dates)
- Notification preferences
- Security activity summary
- Consents granted

[Request export]

Modal: Export in progress

Your data export is being prepared.

Estimated time: 5-10 minutes

We'll notify you by email when it's ready.

[Close]

Notification (when ready):

✓ Your data export is ready

[Download data] (expires in 7 days)

4.3.2 Delete Account Section

UI:

Delete Your Account

This action is permanent and cannot be undone.

Before deleting:

- Transfer ownership of any workspaces you own
- Download your data if needed
- Cancel active subscriptions

[Delete account]

Modal: Delete account?

⚠ Warning: This action is permanent

You will lose access to:

- All workspace memberships
- All app data (Inventory, Task Management)
- Your profile and settings

Business records will be preserved:

- Your employer's inventory data will not be deleted
- Transaction history will be retained

Type "DELETE" to confirm: [DELETE]

Reason (optional): [No longer needed v]

[Cancel] [Request deletion]

Banner (after request):

⚠ Your account will be deleted on Sep 16, 2026. [Cancel deletion]

4.4 Workspace Memberships Page

UI:

Your Workspaces

You belong to 2 workspaces.

Workspace	Role	Products	Joined	Actions
Code X Stores	Owner	Inventory, Task Mgmt	Jan 2026	[Manage]
Orviohub Team	Member	Task Management	Mar 2026	[Leave]

Modal: Leave workspace?

Leave "Orviohub Team"?

You will lose access to:

- All Task Management data in this workspace
- Team conversations and projects

This cannot be undone.

[Cancel] [Leave workspace]

Error (if only owner):

You cannot leave "Code X Stores" because you're the only owner.

Transfer ownership first:
[Transfer ownership] [Delete workspace]

5. Security & Compliance

5.1 Re-authentication Requirements

Sensitive actions requiring current password or recent auth:[\[6\]](#) [\[3\]](#) [\[5\]](#)

Action	Re-auth Method	Rationale
Change password	Current password	Security boundary, prevents session hijacking [12]
Change email	Current password	High-risk change, prevents account takeover [1]
Enable 2FA	Current password	Security enhancement, requires proof of identity [4]
Disable 2FA	Current password + TOTP code	Critical security change, dual verification [4] [6]
Delete account	Current password	Destructive action, prevents accidental deletion [1]
Link OAuth provider	Current password	Prevents unauthorized account linking [3]
Unlink last OAuth provider	Current password	Prevents account lockout [16]

5.2 Rate Limiting

Endpoint	Limit	Window
Password change	5 attempts	1 hour
2FA verification	5 attempts	15 minutes
Email change	3 requests	24 hours
Data export	1 request	7 days
Account deletion	1 request	30 days
Session revocation	10 requests	1 minute

5.3 Audit Logging

All profile changes logged to `userAuditLogs`: [\[17\]](#) [\[1\]](#)

Event Type	Severity	Metadata Fields
<code>password.changed</code>	high	<code>ipAddress</code> , <code>userAgent</code>
<code>2fa.enabled</code>	high	<code>ipAddress</code> , <code>method: "totp"</code>
<code>2fa.disabled</code>	high	<code>ipAddress</code> , <code>reason</code>
<code>email.change_requested</code>	high	<code>oldEmail</code> , <code>newEmail</code>
<code>email.changed</code>	high	<code>oldEmail</code> , <code>newEmail</code>
<code>session.revoked</code>	medium	<code>sessionId</code> , <code>deviceId</code> , <code>reason</code>
<code>session.revoked_all</code>	medium	<code>revokedCount</code>
<code>account.deletion_requested</code>	critical	<code>reason</code> , <code>scheduledDeletionAt</code>
<code>account.deleted</code>	critical	<code>preservedDataTypes</code>
<code>data.export_requested</code>	medium	<code>exportId</code>
<code>oauth.linked</code>	high	<code>provider</code> , <code>providerEmail</code>
<code>oauth.unlinked</code>	high	<code>provider</code>

5.4 NDPR/GDPR Compliance

Data subject rights (Nigeria Data Protection Act 2023): [\[7\]](#) [\[1\]](#)

-  **Right to be informed:** Privacy policy, data summary page
-  **Right of access:** Data export feature (JSON/CSV)
-  **Right to rectification:** Profile editing (name, email, phone)
-  **Right to erasure:** Account deletion (with business record preservation)

- ✓ **Right to restrict processing:** Marketing email opt-out
- ✓ **Right to data portability:** Structured export format (JSON)
- ✓ **Right to object:** Unsubscribe from marketing communications

Data retention:

- **Personal data:** Deleted on account deletion (anonymized, not fully removed)
- **Business records:** Preserved indefinitely (transactions, audit logs)
- **Security logs:** Retained for 2 years (compliance requirement)

6. Testing Plan

6.1 Unit Tests

```
// 2FA setup
test('generates valid TOTP secret', async () => {
  const secret = generateTotpSecret('code@example.com');
  expect(secret.base32).toMatch(/^[A-Z2-7]{32}$/);
  expect(secret.otppath_url).toMatch(/otppath:\//otppath/);
});

test('verifies TOTP code within time window', async () => {
  const secret = 'JBSWY3DPEHPK3PXP';
  const token = speakeasy.totp({ secret, encoding: 'base32' });
  const valid = verifyTotp(secret, token);
  expect(valid).toBe(true);
});

test('rejects expired TOTP code', async () => {
  const secret = 'JBSWY3DPEHPK3PXP';
  const oldToken = '123456'; // expired
  const valid = verifyTotp(secret, oldToken, { window: 0 });
  expect(valid).toBe(false);
});

// Password change
test('changes password and revokes sessions', async () => {
  const user = await createUser({ passwordHash: bcrypt.hashSync('OldPass123!', 12) });
  await createSession({ userId: user.id, sessionHash: 'session_1' });
  await createSession({ userId: user.id, sessionHash: 'session_2' });

  await changePassword(user.id, 'OldPass123!', 'NewPass456!');

  const updated = await getUser(user.id);
  expect(bcrypt.compareSync('NewPass456!', updated.passwordHash)).toBe(true);

  const sessions = await getSessions(user.id);
  expect(sessions.filter(s => !s.revokedAt).length).toBe(1); // only current session
});

// Email change
```

```

test('sends verification email on email change request', async () => {
  const user = await createUser({ email: 'old@example.com' });
  const mockSendEmail = jest.fn();

  await requestEmailChange(user.id, 'currentPass123!', 'new@example.com', mockSendEmail);

  expect(mockSendEmail).toHaveBeenCalledWith(
    'new@example.com',
    expect.objectContaining({ subject: 'Verify your new email - Orv.iohub' })
  );
});

// Account deletion
test('prevents deletion if user owns workspace', async () => {
  const user = await createUser();
  await createWorkspace({ ownerId: user.id, name: 'Test Workspace' });

  await expect(requestAccountDeletion(user.id, 'DELETE')).rejects.toThrow(
    'You own 1 workspace. Transfer ownership before deleting your account.'
  );
});

test('anonymizes user data on deletion', async () => {
  const user = await createUser({ email: 'test@example.com', firstName: 'Test' });
  await deleteAccount(user.id);

  const deleted = await getUser(user.id);
  expect(deleted.email).toMatch(/deleted_usr_\w+@orv.iohub\.local/);
  expect(deleted.firstName).toBeNull();
  expect(deleted.status).toBe('deleted');
});

```

6.2 Integration Tests

```

// Full 2FA enablement flow
test('end-to-end 2FA setup and verification', async () => {
  const user = await loginUser();

  // Start 2FA setup
  const startResponse = await api.post('/v1/users/me/security/2fa/start');
  expect(startResponse.status).toBe(200);
  expect(startResponse.body.secretUrl).toMatch(/otpauth:\/\/totp/);
  expect(startResponse.body.backupCodes).toHaveLength(10);

  // Verify TOTP code
  const secret = extractSecretFromUrl(startResponse.body.secretUrl);
  const totpCode = speakeasy.totp({ secret, encoding: 'base32' });
  const verifyResponse = await api.post('/v1/users/me/security/2fa/verify', { totpCode });
  expect(verifyResponse.status).toBe(200);

  // Check 2FA enabled
  const statusResponse = await api.get('/v1/users/me/security/2fa/status');
  expect(statusResponse.body.totpEnabled).toBe(true);
  expect(statusResponse.body.backupCodesRemaining).toBe(10);
});

```

```

    // Verify audit log
    const auditLogs = await db.userAuditLogs.by_user({ userId: user.id });
    expect(auditLogs.some(log => log.eventType === '2fa.enabled')).toBe(true);
  });

  // Email change flow
  test('end-to-end email change with verification', async () => {
    const user = await loginUser({ email: 'old@example.com' });

    // Request email change
    const changeResponse = await api.post('/v1/users/me/email/change', {
      currentPassword: 'TestPass123!',
      newEmail: 'new@example.com'
    });
    expect(changeResponse.status).toBe(200);

    // Simulate email verification
    const token = generateVerificationToken();
    const verifyResponse = await api.post('/v1/users/me/email/verify', { token });
    expect(verifyResponse.status).toBe(200);

    // Check email updated
    const updatedUser = await db.users.get(user.id);
    expect(updatedUser.email).toBe('new@example.com');
    expect(updatedUser.emailVerifiedAt).toBeDefined();

    // Check sessions revoked (except current)
    const sessions = await db.userSessions.by_user({ userId: user.id });
    expect(sessions.filter(s => !s.revokedAt).length).toBe(1);
  });

  // Data export flow
  test('request and download data export', async () => {
    const user = await loginUser();

    // Request export
    const exportResponse = await api.post('/v1/users/me/data-export');
    expect(exportResponse.status).toBe(200);
    expect(exportResponse.body.status).toBe('processing');

    // Simulate background job completion
    await processDataExport(exportResponse.body.exportId);

    // Check export completed
    const statusResponse = await api.get(`/v1/users/me/data-export/${exportResponse.body.exportId}`);
    expect(statusResponse.body.status).toBe('completed');
    expect(statusResponse.body.downloadUrl).toBeDefined();

    // Download data
    const downloadResponse = await api.get(statusResponse.body.downloadUrl);
    expect(downloadResponse.status).toBe(200);
    expect(downloadResponse.headers['content-type']).toBe('application/zip');
  });

```

6.3 Security Tests

```
// Rate limiting
test('rate-limits password change attempts', async () => {
  const user = await loginUser();

  for (let i = 0; i < 5; i++) {
    await api.post('/v1/users/me/password/change', {
      currentPassword: 'WrongPass!',
      newPassword: 'NewPass123!'
    });
  }

  const sixthAttempt = await api.post('/v1/users/me/password/change', {
    currentPassword: 'WrongPass!',
    newPassword: 'NewPass123!'
  });
  expect(sixthAttempt.status).toBe(429); // Too Many Requests
  expect(sixthAttempt.body.error).toBe('Too many attempts. Try again in 1 hour.');
```

```
});

// Session revocation
test('revoked session cannot be used', async () => {
  const user = await createUser();
  const session1 = await createSession({ userId: user.id });
  const session2 = await createSession({ userId: user.id });

  // Revoke session2
  await api.withSession(session2).post('/v1/users/me/sessions/revoke-all');

  // Try to use revoked session
  const response = await api.withSession(session2).get('/v1/users/me');
  expect(response.status).toBe(401); // Unauthorized

  // Current session still works
  const currentResponse = await api.withSession(session1).get('/v1/users/me');
  expect(currentResponse.status).toBe(200);
});

// Account enumeration prevention
test('does not reveal if email exists on deletion request', async () => {
  const response1 = await api.post('/v1/users/me/deletion-request', {
    confirmationText: 'DELETE'
  });

  // Create new account with different email
  const newUser = await createUser({ email: 'other@example.com' });
  const response2 = await api.withUser(newUser).post('/v1/users/me/deletion-request'
    confirmationText: 'DELETE'
  });

  // Both responses should be identical (no enumeration)
  expect(response1.body.scheduledDeletionAt).toBeDefined();
  expect(response2.body.scheduledDeletionAt).toBeDefined();
```



```
// Response times should be similar (prevent timing attacks)
});
```

7. Implementation Phases

Phase 1: Profile Foundation (Week 1)

Deliverables:

- ✓ Convex schema: `users`, `userPreferences`, `userAuditLogs` (enhanced) ^[18] ^[17]
- ✓ API: `GET/PATCH /v1/users/me`, `POST/DELETE /v1/users/me/avatar`
- ✓ UI: Personal info page (name, avatar, bio, timezone, language) ^[19] ^[7]
- ✓ Unit tests for profile updates

Success criteria:

- User can view/edit profile
- Avatar upload works (file validation, compression)
- Changes reflected across all workspaces

Phase 2: Contact Management (Week 2)

Deliverables:

- ✓ API: `POST /v1/users/me/email/change`, `POST /v1/users/me/email/verify` ^[20]
- ✓ Email verification flow (token generation, hashing, expiry) ^[13]
- ✓ Phone number support (optional, OTP verification) ^[20]
- ✓ Security notifications (email change emails) ^[1]
- ✓ Integration tests for email change

Success criteria:

- User can change email securely
- Verification email sent + token validated
- Old email notified of change

Phase 3: Security Core (Week 3)

Deliverables:

- ✓ API: `POST /v1/users/me/password/change` (with bcrypt hashing) ^[5]
- ✓ TOTP 2FA: `POST /v1/users/me/security/2fa/start`, `verify`, `disable` ^[4] ^[2]
- ✓ QR code generation (qrcode library) ^[4]

- ✓ Backup codes (generation, hashing, display) ^[10]
- ✓ Session management: GET /v1/users/me/sessions, revoke endpoints ^{[21] [22]}
- ✓ Security activity log: GET /v1/users/me/security-activity ^{[22] [23]}
- ✓ Security tests (rate limiting, session revocation)

Success criteria:

- User can enable 2FA with Google Authenticator
- Password change revokes other sessions
- Active sessions list shows device info
- All security events logged

Phase 4: Preferences & Notifications (Week 4)

Deliverables:

- ✓ API: GET/PATCH /v1/users/me/preferences ^[24]
- ✓ API: GET/PATCH /v1/users/me/notifications/preferences ^{[23] [25]}
- ✓ UI: Notification preferences page (security, workspace, product, billing, marketing) ^[23]
- ✓ UI: Preferences page (theme, language, timezone, date/number formats) ^[26]
- ✓ Theme switching (light/dark/system)

Success criteria:

- User can toggle email/in-app notifications per category
- Security emails cannot be disabled
- Theme/language preferences persist

Phase 5: Workspace Memberships (Week 5)

Deliverables:

- ✓ API: GET /v1/users/me/workspaces ^{[27] [26]}
- ✓ API: POST /v1/users/me/workspaces/:workspaceId/leave ^[27]
- ✓ UI: Workspace memberships page (list, roles, leave workspace) ^[27]
- ✓ Owner protection logic (cannot leave if only owner) ^{[28] [27]}
- ✓ Integration tests for workspace leave flow

Success criteria:

- User sees all workspace memberships
- Can leave workspace (if not only owner)
- Clear error if ownership transfer required

Phase 6: Privacy & Data Rights (Week 6)

Deliverables:

- ✓ API: GET /v1/users/me/data-summary, POST /v1/users/me/data-export ^[29] ^[15]
- ✓ Background export job (JSON/CSV generation, ZIP packaging)
- ✓ API: POST /v1/users/me/deletion-request, cancel ^[15]
- ✓ Background deletion job (anonymization, business record preservation) ^[15]
- ✓ UI: Privacy page (data export, delete account, consents) ^[29]
- ✓ NDPR compliance audit (data rights, retention policy) ^[7] ^[1]

Success criteria:

- User can request data export (downloadable ZIP)
- Account deletion has 7-day cooling-off period
- Business records preserved on deletion

Phase 7: Hardening & Polish (Week 7)

Deliverables:

- ✓ Security audit (penetration testing, OWASP ASVS checklist) ^[30] ^[^31] ^[21]
- ✓ Performance optimization (index usage, query caching) ^[^32] ^[17] ^[19]
- ✓ Accessibility audit (WCAG 2.1 AA compliance)
- ✓ Error handling (user-friendly messages, graceful failures)
- ✓ Documentation (API docs, user guides, privacy policy)
- ✓ Load testing (1000 concurrent profile updates)

Success criteria:

- All security tests pass
- No critical vulnerabilities
- Profile pages load in <500ms
- Docs complete

8. Risks & Mitigations

Risk	Impact	Mitigation
TOTP secret leakage	High	Encrypt secrets at rest (AES-256-GCM), never log secrets ^[8] ^[4] ^[9]
Session hijacking	High	Re-authentication for sensitive actions, session invalidation on password change ^[3] ^[14]

Risk	Impact	Mitigation
Email enumeration	Medium	Same response time/message for "email exists" vs "not found" on deletion ^[12] ^[13]
Data export abuse	Low	Rate limit: 1 export per 7 days, pre-signed URLs expire in 7 days ^[1]
Accidental account deletion	Critical	7-day cooling-off period, explicit confirmation ("DELETE" typing), ownership checks ^[15]
Business record loss	Critical	Deletion job preserves workspace data, only anonymizes personal fields ^[1] ^[15]

9. Success Metrics

Track post-launch:

- **2FA adoption rate:** Target >40% of active users within 3 months ^[2]
- **Email change success rate:** >95% completion (verification email clicked)
- **Data export requests:** <5% of users/month (monitor for abuse)
- **Account deletion rate:** <1% of users/month (investigate spikes)
- **Support tickets:** <10 tickets/week related to profile/security
- **Security email open rate:** >60% (password change, 2FA enabled notifications)

This implementation plan provides a complete, production-ready specification for Orviahub's personal profile system, covering security, contact management, preferences, workspace memberships, and privacy controls aligned with NDPR requirements. ^[2] ^[3] ^[4] ^[5] ^[1]



1. docs/User Personal Profile Management Plan.pdf
2. <https://shattered.io/two-factor-authentication-nodejs/>
3. <https://deepwiki.com/OWASP/CheatSheetSeries/3-authentication-and-session-management>
4. <https://www.boxsoftware.net/how-to-implement-two-factor-authentication-in-a-node-js-app-with-totp/>
5. <https://deepwiki.com/OWASP/CheatSheetSeries/3.1-password-storage-and-hashing>
6. <https://deepwiki.com/OWASP/CheatSheetSeries/3.4-account-recovery-and-security-questions>
7. <https://nodefunc.com/blogs/two-factor-authentication-2fa-implementation-guide>
8. <https://tech-insider.org/au/how-to-add-google-authenticator-2fa-to-your-app-2026/>
9. <https://www.promptstoproduct.com/how-to-add-2fa-to-saas>
10. <https://stackinsight.dev/blog/2fa-totp-implementation>
11. <https://www.monito.dev/blog/test-password-reset-flow>
12. <https://coldletter.com/blog/password-reset-email/>
13. <https://www.training-central.net/2026/07/07/owasp-session-management-cheat-sheet/>
14. <https://codeables.dev/article/how-do-saas-companies-handle-parent-child-accounts-for-enterprises>
15. docs/Workspace Management Implementation Plan.pdf

16. <https://old.sectionvtrack.com/article/how-to-add-people-to-github-repo-the-definitive-process>
17. <https://www.techbloat.com/how-to-give-access-to-github-repository.html>
18. <https://docs.github.com/en/rest/orgs/members?apiVersion=2026-03-10>
19. <https://tessl.io/registry/skills/github/better-auth/skills/organization-best-practices>
20. <https://tessl.io/registry/skills/github/slurpyb/skills/organization-best-practices--organization-best-practices>
21. <https://sociilabs.com/blog/saas-multi-tenancy-architecture>
22. <https://github.com/orgs/community/discussions/188479>
23. <https://note.com/abc1nc/n/n1cdd881e4b38?hl=en>
24. <https://www.stitchflow.com/user-management/github/manual>
25. <https://www.scribd.com/document/978888905/GitHub-Onboarding-and-Licensing-Guide>
26. https://note.com/fast_owl436/n/n8970392284fe?hl=en
27. <https://docs.github.com/en/rest/teams/members>
28. <https://www.sent.dm/en/resources/code-examples/twilio-node-js-next-js-otp-2fa>
29. <https://oneuptime.com/blog/post/2026-01-30-security-multi-factor-authentication/view>
30. <https://www.gitnexus.com/blogs/two-factor-authentication>